

DDL, DML, SQL

Prof:

Francesco SANNA

Table of Contents

DDL	3
DDL_01	3
DDL_02	3
DDL_03	3
DDL_04 – DDL_05	3
DDL_06 - Authors	3
DDL_07 Books.....	3
DDL_08 Company	4
DDL_09 Train Table	4
DDL_10 Shop	4
DDL_11 Teams.....	4
DDL_12 Hospital	5
DDL_13 Jobs	5
DDL_14 Countries.....	5
DML	6
DML_001 – DML_002.....	6
DML_003 - Authors	6
DML_004 – Books.....	6
DML_005 – Teacher	6
DML_006 – Student.....	7
DML_007 – Person	7
DML_008 – Es 6 pag 239	7
DML_009 – Train Station.....	8
DML_010 – Car Rental.....	8
DML_011 – Conference Room Allocation (No solution)	9
SQL.....	10
SQL_001 – Photo Archive	10
SQL_002 – Students	10
SQL_003 – Car Dealer.....	10
SQL_004 – Sales.....	11
SQL_005 – Patients.....	12

DDL

DDL_01

Create, if not exists, a database called *school*.

Destroy, if exists, a database called *school*.

DDL_02

After creating the database *school* (if it does not exist), create the table *students* with the following attributes:

- ID_Student, simple attribute, string of 4 characters, primary key
- name, simple attribute, string of 40 characters,
- address, simple attribute, string of 80 characters

DDL_03

After creating a database with the name *DB_NumericalData*

Analyze the different types of data.

In particular, check the difference between exact numbers:

BIGINT, INTEGER, SMALLINT, TINYINT, BIT, DECIMAL(precision, scale), NUMERIC(precision, scale)

And approximated numbers:

FLOAT, REAL, DOUBLE(numero cifre, decimali)

DDL_04 – DDL_05

Analysis of the key constraints and of the intrarelational referential integrity constraints.

DEFAULT, NULL, NOT NULL, CHECK, UNIQUE

DDL_06 - Authors

In the database called as the exercise, write the SQL instructions to create a table which corresponds to the following relational scheme:

author(code, cf, surname, name, city)

inserting for each attribute eventually the constraint.

DDL_07 Books

For the database called as the exercise, write the SQL instructions to create a table which corresponds to the following relational scheme:

libri(isbn, title, author, topic, editor, year, edition)

inserting for which attribute the right constraint applies.

DDL_08 Company

For the database called as the exercise, write the SQL instructions for the creation of the following tables, assigning the correct format of the attributes:

```
employee(ID_employee, surname, name, DOB, zip_code, city, seniority, id_department)
```

```
department(ID_department, name_department, cod_responsible)
```

inserting for each attribute the corresponding constraint. Do not put (for now) in relation the tables.

Note: It is good habit to make sure that the DOB are after 1900 and that seniority cannot be negative

DDL_09 Train Table

For the database called as the exercise, create the tables in SQL assigning the correct format for the attributes.

```
train(number, day, hour, destination, category)
```

```
stop(train, station, hour)
```

No null values are admitted for the attributes. Insert referential integrity between trains and stops.

There cannot be two trains with the same hour, same destination and same category.

DDL_10 Shop

For the database Shop_Exercise, create the following tables in SQL assigning the correct format for the attributes:

```
client(ID_client, surname, name, city, discount)
```

```
agent(ID_agent, surname, name, zone, supervisor, commission)
```

```
order(num_order, id_client, id_agent, date_order, product, amount)
```

inserting for each attribute the relative inter-referential integrity constraint.

DDL_11 Teams

For the database Teams_Exercise create the following tables in SQL assigning the correct format for the attributes:

```
team(ID_team, name, city, id_stadium)
```

```
stadium(ID_stadium, name, city)
```

```
match(day, team_house, team_host, goal_TeamHome, goal_TeamHost)
```

```
ranking(day, team, score, position)
```

Null attributes are not admitted, and teams must be different in the entity match. It is requested that referential integrity constraints are requested among:

- team and match,
- team and stadium,
- ranking and team.

Should a team be cancelled, it is necessary to foresee that all matches the team played are cancelled.

DDL_12 Hospital

For the database Hospital_Exercise, the following tables in SQL need to be created, assigning the correct format for the attributes.

pazient(code, surname, name, healthy_number, DOB)
 doctor(ID_doctor, surname, name, department, specialization)
 specialization(description)
 department(code, description, name, chief_doctor)

No null values are admitted for the following attributes:

- surname and name of the patients,
- name of the departments,
- surname and name of doctors,
- chief doctor of the department,

Moreover, the following referential constraints are requested:

- departments and specializations

DDL_13 Jobs

Create the table:

job(job_ID, JOB_TITLE, MIN_SALARY, MAX_SALARY)

Make sure that the maximum salary is less or equal to 25000.

DDL_14 Countries

Create the following table:

country(COUNTRY_ID, COUNTRY_NAME, REGION_ID)

Make sure the only admitted names are Italy, India, e China.

DML

DML_001 – DML_002

Theory

DML_003 - Authors

After creating the table in agreement with the relational scheme:

```
author(code(pk), nat_ID, surname, name, city)
```

inserting the key constraint for the primary key, write the following queries:

- rename the attribute nat_id into national_id;
- add the attribute address of dimension of 40 characters;
- add the attribute zip_code of dimension of 5 characters;
- add the constraint NOT NULL to the attribute name;
- add the constraint NOT NULL to the attribute name, varying the field to 15 characters;

DML_004 – Books

After creating the table which corresponds to the following relational scheme:

```
book(isbn(pk), title, author, topic, editor, year, edition)
```

inserting the key constraint for the primary key, write the queries to:

- rename the attribute year to publication_year;
- add the attribute surname with dimension of 40 characters;
- add the attribute name with dimension of 40 characters;
- delete the attribute author;
- rename the attribute surname in surname_author;
- rename the attribute name in author_name;
- add the constraint NOT NULL to the attributes author_surname, author_name.

DML_005 – Teacher

After creating the table which corresponds to the following relational scheme:

```
teacher(code(pk), nat_ID, surname, name, city, telephone, habilitation)
```

inserting the primary key constraint for the code, write the queries to:

- rename the attribute habilitation into habilitation_class;
- rename the attribute nat_ID into fiscal_ID;
- add the attribute role as boolean;
- delete the attribute city;
- delete the attribute telephone;
- add the constraint NOT NULL to the attribute habilitation_class

DML_006 – Student

After creating the table which corresponds to the following relational scheme:

student(code(pk), Nat_ID, surname, name, date, class, telephone)

inserting the primary key constraint for the code, write the following queries:

- rename the attribute Nat_ID into fiscal_code;
- rename the attribute date into DOB;
- add the attribute course with dimension 20 characters;
- add the attribute Section with a dimension of 5 characters;
- rename the attribute course in school_address;
- eliminate the attribute telephone.

DML_007 – Person

After creating a table which corresponds to the following national schemes:

person(card_number(pk), fiscal_code, surname, name, DOB, pathology, exemption, allergy)

where null values are not admitted for the first 4 fields, write the following queries:

- insert 5 records, 3 of which they need to have all data;
- delete the records which have "E48" as the exemption code;
- modify all the anagraphic data, by transforming them in capital letters;
- insert as exemption "for age" to the people with more than 65 years.

DML_008 – Es 6 pag 239

After creating a table which corresponds to the following relational scheme:

disk(number(pk), artist, title, label, year, country, format, disk_conditions, cover_conditions, min_price, max_price, new_price)

write the queries,

knowing that conditions could be 1. poor/fair, 2. good/good plus, 3. very good, 4. very good plus, 5. near mint, 6. mint, and that the format could be '45 RPM' or '78 RPM':

- insert 10 disks, where 3 have data for each attribute;
- add the min price of € 10 to the disks without a minimum price;
- delete the disks where the maximum price is lower than minimum price;
- increase the new_price of 10% for those disks with more than 10 years;
- diminish the minimum price by 5% for disks of the last year;
- increase the maximum price by 10% for those disks with both disk and cover conditions equal to 'mint';
- delete disks with '78 RPM'.

Data - Proposal:

Artist	Title	Label	Year	Country	Format	Disk Conditions	Cover Conditions	Min Price	Max Price	New Price
Ligabue	Certe Notti	CN	2003	Italia	45 RPM	Near Mint	Very Good	30	45	55.6
Queen	We are the Champions	Best of the Queen	1989	USA	78 RPM	Poor/Fair	Near Mint	58.7	41.6	68.9
Oasis	Wonderwall	Morning Glory	1995	UK	78 RPM	NULL	Min	NULL	40.8	53.1
Madonna	Frozen	The Witch Morgana	1998	USA	45 RPM	Very Good	Mint	28.9	41.7	52.4
Fedez	Disco Paradise	Disco Paradise	2023	Italia	45 RPM	Mint	Good/Good Plus	27.8	40.4	51.6
Fedez	Margarita	This is Elodie	2019	NULL	78 RPM	Poor/Fair	Near Mint	18.7	37.6	50.1
RHCP	Californication	NULL	2019	USA	78 RPM	Very Good	NULL	NULL	37.2	49.42

DML_009 – Train Station

After creating the following table as in the relational scheme:

```
train(number, day, hour, destination, category)
station(ID_station, station_name)
stop(id_station, train_num, hour)
```

where null values are admitted and, among referential integrity constraints between train and stops, it is not possible to have two trains with the same time, same destination and same category. Write the following queries to:

- insert 6 stations on a desired line;
- insert 3 trains, one for each category ('Intercity', 'direct', 'local');
- insert for each train the corresponding stop, knowing that the intercity makes only one intermediate stop

Then update the summer time with the winter time, moving all trains of one hour, with exception for the intercity.

DML_010 – Car Rental

After creating the corresponding tables associated to the relational scheme for a car renting company:

```
rent(ID_rent, dates, hour, id_client, id_car, duration, checkin_location, checkout_location, price)
```

```
car(ID_car, brand, model, color, km, plate, category, price)
```

```
client(ID_client, surname, name, address, age, driving_licence)
```

where the categories are:

1. small, 2. medium, 3. big(monovolume), 4. suv

Moreover, null values are not admitted. With respect to referential integrity constraints between rents and cars, we cannot have two cars with the same plate, and two rents for the same date for the same client.

Please write the queries to:

- insert 10 cars, at least 2 for each category;
- insert 3 clients;
- insert for each car at least 3 rents;
- add for cars another category "5. luxury", and add 3 rents;

- update the price of cars of category 5, increasing the price of 10%;
- update the price of cars of category 1, decreasing it by 15%;
- delete the rents of 2021 for cars whose price rent is less than 100.

DML_011 – Conference Room Allocation (No solution)

After creating the tables which correspond to the following relational scheme for a system which keep tracks of rooms in a congressional center:

Room(ID_room, NumbPlaces_Max, Available (Y/N), Surface)

Allocation(NR_sessione, id_room, occupied_seats)

Inscription(Person, Numb_Session)

Where 10 sessions are held with a progressive number from 1 to 10 and are allocated to some rooms, numbered between 1 and 10, with capacity up to 50 people.

Please write the queries to:

- assign sessions to rooms, in such a way that the room of the session number cannot correspond to the number of rooms;
- rename the available attribute available with occupied_seats;
- Increase the capacity of 10% for rooms with a surface greater than 120 squared meters;
- Set the capacity of each room to the half in case of COVID emergency, verifying also the number of occupied seats;
- Delete the allocation of a session if the number of occupied seats is less than 2

SQL

SQL_001 – Photo Archive

we want to represent a database for a photo archive where photos are catalogued based on the subject they represent, of the location, of the dimension and whether they are color or B/W.

Photos could describe people, locations or objects.

A possible relational scheme is the following:

PHOTO(ID_photo, date, dimension, state, type, subject, location)

The state can be old, recent, or new.

Create the SQL queries necessary to execute the following tasks:

- list all color photos
- List all color photos in Milan
- List photos with the city of Milan
- List all photos with the pope in visit in Naples
- List all photos of Rome in 2000
- List all color photos of dimensions 5 x 12 cm

SQL_002 – Students

You are working for an Italian high school. Italian high schools have 5 years.

Create a table named *Students* with the following structure:

Students(ID_Student, surname, name, address, class, section, grade_average, adult)

The field **adult** could be "Y" or "N", depending on whether the student is adult or not.

Create the SQL queries necessary to execute the following tasks:

- List all students of different classes who are already adult (age greater than 18)
- List all students of 4A
- List all students who have the average between 6 and 7, but who are not part of class 4th and 5th
- List all students with an average between 5 and 6 and between 8 and 9
- List all students of all classes 4th and 5th
- List all students whose surname begins with "I" and with "S".

SQL_003 – Car Dealer

A car dealership sells cars new and used.

Relational Scheme:

car(ID_car, brand, model, type, price, registration_date, color, km, waiting_days, new_used, seller)

Attribute constraints:

- Price greater than 0
- New_Used must have the two choices (New or Used), and as default it must have 'used'

- Registration date must be after June 15th, 1920
- Mandatory fields: brand, model, registration_date

Create the SQL queries to execute the following tasks:

- List all new cars immediately available for sale
- List all new cars of Fiat
- List all used cars with less than 100000 km
- List all cars with color red and cost less than €10.000
- List all cars sold to Mr. James Bond
- List all the Fiat cars with waiting lists less than 30 days
- List all cars with price less than €15.000 of color red
- List the pandas used available with less than 200.000 km

SQL_004 – Sales

You are given the following relational scheme. S are suppliers, P the products and SP is the table which provides the quantity *Quant* of the product *P* provided by Supplier *S*

P(CodP, NameP, Colour, Size, Storage)

S(CodS, NameS, NSociet, Location)

SP(CodP, CodS, Quant)

The data provided is :

S

CodS	NameS	NSociet	Location
S1	Andrew	2	Milan
S2	Lydia	1	Beijing
S3	Gabriel	3	Paris
S4	Antoine	2	Rome
S5	John	3	London

P

CodP	NameP	Colour	Size	Storage
Prod1	T-shirt	Red	40	Shanghai
Prod2	Pants	Green	48	Milan
Prod3	Shirt	Blue	44	Rome
Prod4	Shirt	Blue	40	Paris
Prod5	Skirt	Blue	44	London

SP

CodP	CodS	Quant
Prod1	S1	300
Prod1	S2	200
Prod1	S3	400
Prod1	S4	200
Prod1	S5	100
Prod2	S1	300
Prod2	S2	400
Prod3	S2	200
Prod4	S3	200

SQL_004_01

Write the following queries:

- Find the code and the number of societies of suppliers of Milan
- Find the code of all products
- Find the code of the products supplied by at least a supplier
- Find all information about products
- Find the code of the products and the size expressed in the American form (reduced by 14)
- Find the code of the suppliers of Milan
- Find the code of number of societies of the suppliers which do not have a location in Milan
- Find the code of number of societies of the suppliers which have a location in Milan and the number of Societies is greater than 2.
- Find the number of the suppliers which supply the products Prod2

SQL_004_02

Write the following queries:

- Find all the information about the products ordering the result ascending by name and descending by size
- Find all information about products ordering ascending by USA size
- Find the code and the name of the products whose name starts with P
- The attribute Storage is precisely Paris
- Select the products whose storage name contains an n
- Select the products whose storage name where the 3rd letter is an n
- Select the products whose storage name does NOT CONTAIN an i in second position

SQL_004_03

Write the following queries:

- Find the name of the suppliers which supply at least a red product
- Find the couples of suppliers codes such that
- both suppliers have the same location
- Find the number of suppliers
- Find the number of suppliers which have supplied at least one product
- Count how many suppliers have supplied Prod2
- Calculate the total quantity of product Prod2 supplied.
- For each product, find the total quantity supplied.

SQL_004_04

Write the following queries:

- For each product, find the code and the total quantity supplied by suppliers of Milan.
- For each product, find the code and the name, as well as the total quantity supplied.

SQL_005 – Patients

The following tables are provided:

patient(ID_patient, surname, name, DOB, province, hospital_code)

visit(ID_visit, visit_date, weight, height, min_pressure, max_pressure, id_patient, blood_glucose_test)

Insert some data into the two tables.

Create the necessary queries to execute the following tasks:

- List all patients who do not have the hospital_code
- List all visits: visit date, surname and weight measure at the time of the visit
- List all patients who have a difference between max pressure and min pressure smaller than 40 mmHg
- List the patients who weigh between 60 and 70 kg.
- List all patients of Milan who have a weight greater than 90 kg.
- List all Napolitan (i.e. from Naples) patients who have diabetes problems (blood glucose test > 110) in 2020.
- List all patients of Rome who are hypertensive (pressure greater than 140) with weight smaller than 60 kg.
- List all patients in alphabetic order who have done the medical visitation this year

SQL_006 – Bicycles

Create a database names bicycles composed by two tables with the same structure:

items(ID_prod, id_brand, cod_model, name_model, type, colour, dimension)

brand(ID_brand, name, city)

First, add the following data:

Brand

Name	City
Specialized	Milan
Felt	Bern
Focus Bikes	Frankfurt
Campagnolo	Rome
Bianchi	Trieste
Colnago	Florence
Giant	Paris
Bergamont	London
Cannondale	Barcelona
Rocky Mountain	Rome

Items

Id_brand	cod_model	Name_model	Type	Colour	dimension
01	S-Works	Tarmac SL8	Shimano Di2	White	
01	S-Works	Roubaix SL8		Black	700x28
01	Aethos Pro			Red	700x25
02	AR FRD	Ultimate	Red eTap AXS	Black	700x25
02	AR FRD	Ultimate	Dura-Ace Di2	Black	700x25
02	AR FRD	Advanced	Ultegra Di2	Grey	700x25
03	Trail	Vam2 SL	9.0	Black	622x30
03	Trail	Vam2 SL	9.9	Yellow	622x30
03	Trail	Jam2 SL	9.0	Black	622x30
03	Trail	Jam2 SL	9.9	Grey	622x30

04	T1	Sea	Climb	Red	620x31
04	T2	Hill	Tourism	Red	620x31
04	T3	Plane	City	Biolet	622x35
04	T4	Mountain	City	Purple	622x35
05	C1	Sea	Run	Red Ferrari	620x31
05	Tu1	Hill	Tourism	Rubin	620x31
05	T3	Plane	Tandem	Grey pencil	622x35
05	Tan2	Hill	Tandem	Pelvinca	622x35
06	Run2	Hill	Run	Yellow light	620x31
06	Tan3	Mountain	Tandem	Dark Blue	620x31
06	Tur3	Mountain	Tourism	Light Blue	620x31
06	Run3	Mountain	Run	Light violet	620x35
07	Ram2	Hill	Climb	Yellow light	620x31
07	Spo3	Mountain	Sport	Dark Blue	620x31
07	Tur3	Mountain	Tourism	Light Blue	620x31
07	Run3	Mountain	Run	Cyan	620x35

Create the following queries

- List all types of bicycles present in the storage
- List all bicycles produced by Campagnolo and whose type is "Tourism".
- List all bicycles produced by "Bianchi" and "Colnago", type "Run" and "tandem".
- List all producers who have produced at least a bicycle type "climb" and "sport".
- List all bicycles produced by the producers "Giant" and "Felt".
- List all producers which have produces with the cod model which starts by "T" and "R".